In-Class Board Problems

Student answer sheets. Algorithms & Data Structures, Professor Roger Hunt

Use one sheet for each problem. Solve the problem at the board in chalk first, and then copy your Python solution onto the sheet and fill in the analysis below it.

1. Phones and computers are locked away for the first half of class, while you work at the board, and you work on your own.
2. Write a plain Python function. You may use anything built into Python, plus `deque` from the `collections` module and the `heapq` module.
3. State the time complexity and the auxiliary space in Big-O. Auxiliary space is the extra memory your solution uses beyond its input, and anything your solution builds and returns counts toward it.
4. Estimate the memory for the input size printed on the sheet. Give the memory that the input uses, the memory that your own data uses, and the total, in bytes, and write down your assumptions.
5. If you get stuck, you may ask for help once you have made two attempts that did not work. Write both attempts at the bottom of the sheet before you ask.
6. A correct answer earns 50 points of extra credit, which makes up for missed classes. An answer is correct when the solution works, the time and space are right, and the memory estimate is within about 30 percent of the key.

Memory sizes in CPython 3.13 on a 64-bit machine

Object	Approximate size
int	28 bytes below 2^{30}, about one billion. The integers from −5 to 256 are shared and cost nothing extra.
float	24 bytes.
True, False and None	Shared, so only the 8-byte pointer to them counts.
str	41 bytes plus 1 byte per character for plain ASCII text. A one-character string is shared.
list	56 bytes plus 8 bytes per slot. A list built with `append` keeps up to about 13 percent spare slots, about 6 percent on average.
tuple	40 bytes plus 8 bytes per item.
dict	64 bytes when empty. With integer keys, about 37 bytes per entry at 1,000 entries, 30 at 10,000 and 52 at 100,000.
set	At least 216 bytes. About 33 bytes per element at 1,000 elements and 42 at 100,000.
deque	About 760 bytes plus 8.3 bytes per element.
node object	About 90 to 100 bytes, plus 28 bytes for a value outside −5 to 256.
recursive call	About 150 to 250 bytes for each call that has not yet returned.

A list holds pointers only, so count the objects it points to separately. A dict or a set briefly holds about 1.5 times its final table while it grows. KB means 1,024 bytes and MB means 1,048,576 bytes.

Node classes

LeetCode defines the node classes for the linked-list and tree problems as follows. You may use them without writing them out.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Name _____ Date _____ Class 1, Problem A

Class 1: Big-O, Big-Omega, Big-Theta and Python memory. Problem A, the easier problem.

#1480 Running Sum of 1d Array

Easy. https://leetcode.com/problems/running-sum-of-1d-array/

You are given a list of integers, `nums`. Return a list in which entry `i` is the sum of `nums[0]` through `nums[i]`. This list is called the running sum.

For example, `nums = [1, 2, 3, 4]` gives `[1, 3, 6, 10]`.

Start with `def runningSum(nums):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 100,000 distinct integers, all above 256, so that none of them is one of Python's shared small integers.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 1, Problem B

Class 1: Big-O, Big-Omega, Big-Theta and Python memory. Problem B, the harder problem.

#217 Contains Duplicate

Easy. https://leetcode.com/problems/contains-duplicate/

You are given a list of integers, `nums`. Return `True` if any value appears at least twice in the list, and `False` if every value is distinct.

For example, `nums = [1, 2, 3, 1]` gives `True`, and `nums = [1, 2, 3, 4]` gives `False`.

Start with `def containsDuplicate(nums):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 100,000 distinct integers, all above 256.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 2, Problem A

Class 2: Arrays and dynamic arrays. Problem A, the easier problem.

#283 Move Zeroes

Easy. https://leetcode.com/problems/move-zeroes/

You are given a list of integers, `nums`. Move every 0 to the end of the list while keeping the non-zero values in their original order. You must change the list in place, without making a copy.

For example, `nums = [0, 1, 0, 3, 12]` becomes `[1, 3, 12, 0, 0]`.

The function returns nothing; it changes `nums` itself. Start with `def moveZeroes(nums):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 100,000 integers, half of them 0. That is ten times LeetCode's limit, which makes the numbers easier to see.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 2, Problem B

Class 2: Arrays and dynamic arrays. Problem B, the harder problem.

#88 Merge Sorted Array

Easy. https://leetcode.com/problems/merge-sorted-array/

You are given two sorted lists. `nums1` has length m + n: its first m entries are values in sorted order, and its last n entries are zeros that serve as empty room. `nums2` holds n values in sorted order. Merge `nums2` into `nums1` so that `nums1` holds all m + n values in sorted order. Change `nums1` in place.

For example, `nums1 = [1, 2, 3, 0, 0, 0]`, `m = 3`, `nums2 = [2, 5, 6]` and `n = 3` turn `nums1` into `[1, 2, 2, 3, 5, 6]`.

The function returns nothing; it changes `nums1` itself. Start with `def merge(nums1, m, nums2, n):`

Your Python solution

Class 2, Problem B, continued: #88 Merge Sorted Array.

Time complexity:

Auxiliary space:

Estimate the memory for m = n = 50,000, so that `nums1` has 100,000 slots and `nums2` has 50,000. The values are distinct integers above 256. LeetCode's limit is m + n ≤ 200; the larger sizes make the numbers easier to see.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 3, Problem A

Class 3: Strings and prefix sums. Problem A, the easier problem.

#14 Longest Common Prefix

Easy. https://leetcode.com/problems/longest-common-prefix/

You are given a list of strings, `strs`. Return the longest string that is a prefix of every string in the list. If no such string exists, return the empty string `""`.

For example, `strs = ["flower", "flow", "flight"]` gives `"fl"`, and `strs = ["dog", "racecar", "car"]` gives `""`.

Start with `def longestCommonPrefix(strs):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for m = 200 strings of 200 characters each, all identical. This is the worst case that LeetCode's limits allow.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 3, Problem B

Class 3: Strings and prefix sums. Problem B, the harder problem.

#303 Range Sum Query - Immutable

Easy. https://leetcode.com/problems/range-sum-query-immutable/

Design a class `NumArray`. It is built once from a list of integers, `nums`. After that, `sumRange(left, right)` is called many times, and each call must return the sum of `nums[left]` through `nums[right]`, including both ends.

For example, after `NumArray([-2, 0, 3, -5, 2, -1])`, the call `sumRange(0, 2)` returns 1, `sumRange(2, 5)` returns −1, and `sumRange(0, 5)` returns −3.

Start with `class NumArray:` and write the methods `__init__(self, nums)` and `sumRange(self, left, right)`.

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 values, which is LeetCode's limit, with values large enough that every prefix sum is a new integer above 256.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 4, Problem A

Class 4: Hash tables. Problem A, the easier problem.

#242 Valid Anagram

Easy. https://leetcode.com/problems/valid-anagram/

You are given two strings, `s` and `t`, made of lowercase letters. Return `True` if `t` is an anagram of `s`, which means that it uses exactly the same letters the same number of times, and `False` otherwise.

For example, `s = "anagram"` and `t = "nagaram"` give `True`, and `s = "rat"` and `t = "car"` give `False`.

Start with `def isAnagram(s, t):`

Your Python solution

Class 4, Problem A, continued: #242 Valid Anagram.

Time complexity:

Auxiliary space:

Estimate the memory for n = 50,000 characters in each string, which is LeetCode's limit.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 4, Problem B

Class 4: Hash tables. Problem B, the harder problem.

#1 Two Sum

Easy. https://leetcode.com/problems/two-sum/

You are given a list of integers, `nums`, and an integer, `target`. Return the indexes of the two numbers that add up to `target`. Exactly one such pair exists, and you may not use the same position twice.

For example, `nums = [2, 7, 11, 15]` and `target = 9` give `[0, 1]`.

Start with `def twoSum(nums, target):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 distinct integers above 256, which is LeetCode's limit, with the matching pair at the very end, so that the dict grows to n − 1 entries.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 5, Problem A

Class 5: Two pointers and sliding windows. Problem A, the easier problem.

#125 Valid Palindrome

Easy. https://leetcode.com/problems/valid-palindrome/

A phrase is a palindrome if, after you turn every uppercase letter into lowercase and remove every character that is not a letter or a digit, it reads the same forward and backward. Given a string `s`, return `True` if it is a palindrome and `False` otherwise.

For example, `s = "A man, a plan, a canal: Panama"` gives `True`, and `s = "race a car"` gives `False`.

Start with `def isPalindrome(s):`

Your Python solution

Class 5, Problem A, continued: #125 Valid Palindrome.

Time complexity:

Auxiliary space:

Estimate the memory for n = 200,000 characters, which is LeetCode's limit, all letters, forming a palindrome. This is the worst case, because the solution checks all the way to the middle.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 5, Problem B

Class 5: Two pointers and sliding windows. Problem B, the harder problem.

#643 Maximum Average Subarray I

Easy. https://leetcode.com/problems/maximum-average-subarray-i/

You are given a list of integers, `nums`, and an integer, `k`. Find the run of k neighboring values that has the largest average, and return that average.

For example, `nums = [1, 12, -5, -6, 50, 3]` and `k = 4` give 12.75, the average of `[12, -5, -6, 50]`.

Start with `def findMaxAverage(nums, k):`

Your Python solution

Class 5, Problem B, continued: #643 Maximum Average Subarray I.

Time complexity:

Auxiliary space:

Estimate the memory for n = 100,000 integers above 256, which is LeetCode's limit. The running sum is a new integer at each step, but only one is alive at a time.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 6, Problem A

Class 6: Stacks. Problem A, the easier problem.

#20 Valid Parentheses

Easy. https://leetcode.com/problems/valid-parentheses/

You are given a string, `s`, that contains only the characters `(`, `)`, `[`, `]`, `{` and `}`. Return `True` if the string is valid and `False` otherwise. A string is valid when every opening bracket is closed by a bracket of the same type, the brackets close in the correct order, and every closing bracket has a matching opening bracket of the same type.

For example, `s = "()[]{}"` gives `True`, while `s = "(]"` and `s = "([)]"` give `False`.

Start with `def isValid(s):`

Your Python solution

Class 6, Problem A, continued: #20 Valid Parentheses.

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 characters, which is LeetCode's limit, all of them `(`. This is the worst case for the size of the stack.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 6, Problem B

Class 6: Stacks. Problem B, the harder problem.

#844 Backspace String Compare

Easy. https://leetcode.com/problems/backspace-string-compare/

Two strings, `s` and `t`, contain lowercase letters and the character `#`, which stands for a backspace: it deletes the character before it, and it does nothing if there is none. Return `True` if the two strings are equal once every backspace has been applied, and `False` otherwise.

For example, `s = "ab#c"` and `t = "ad#c"` give `True`, because both become `"ac"`, while `s = "a#c"` and `t = "b"` give `False`.

Start with `def backspaceCompare(s, t):`

Your Python solution

Class 6, Problem B, continued: #844 Backspace String Compare.

Time complexity:

Auxiliary space:

Estimate the memory for two strings of 200 characters each, which is LeetCode's limit, with no # characters. This is the worst case for the size of the stacks.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 7, Problem A

Class 7: Queues and deques. Problem A, the easier problem.

#933 Number of Recent Calls

Easy. https://leetcode.com/problems/number-of-recent-calls/

Design a class `RecentCounter`. Its method `ping(t)` records a request at time t, measured in milliseconds, and returns how many requests have arrived in the last 3,000 milliseconds, from t − 3000 to t, including this one. Each call uses a larger t than the call before it.

For example, `ping(1)` returns 1, `ping(100)` returns 2, `ping(3001)` returns 3, and `ping(3002)` returns 3.

Start with `class RecentCounter:` and write the methods `__init__(self)` and `ping(self, t)`.

Your Python solution

Class 7, Problem A, continued: #933 Number of Recent Calls.

Time complexity:

Auxiliary space:

Estimate the memory for 10,000 pings, one every millisecond. This is the worst case for the size of the window, which then holds 3,001 entries.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 7, Problem B

Class 7: Queues and deques. Problem B, the harder problem.

#232 Implement Queue using Stacks

Easy. https://leetcode.com/problems/implement-queue-using-stacks/

Implement a first-in, first-out queue using only two stacks. The stacks are Python lists that you may use only with `append`, `pop()` from the end, `[-1]` to look at the end, and `len`. Provide the methods `push(x)`, `pop()`, `peek()` and `empty()`. Every call to `pop` and `peek` is made on a queue that is not empty.

For example, after `push(1)` and `push(2)`, the call `peek()` returns 1, `pop()` returns 1, and `empty()` returns `False`.

Start with `class MyQueue:` and write the methods `__init__(self)`, `push(self, x)`, `pop(self)`, `peek(self)` and `empty(self)`.

Your Python solution

Class 7, Problem B, continued: #232 Implement Queue using Stacks.

Time complexity:

Auxiliary space:

Estimate the memory for 100,000 pushes followed by pops. This is far more than LeetCode's limit of 100 calls, and it makes the growth visible. Assume that each item is a separate integer object that the caller created.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 8, Problem A

Class 8: Linked lists. Problem A, the easier problem.

#206 Reverse Linked List

Easy. https://leetcode.com/problems/reverse-linked-list/

You are given `head`, the first node of a singly linked list. Reverse the list and return its new first node.

For example, the list 1 → 2 → 3 → 4 → 5 becomes 5 → 4 → 3 → 2 → 1.

Each node is a `ListNode` with the fields `val` and `next`. Start with `def reverseList(head):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 5,000 nodes, which is LeetCode's limit. LeetCode's values run from −5,000 to 5,000, so count each value as a new integer.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 8, Problem B

Class 8: Linked lists. Problem B, the harder problem.

#19 Remove Nth Node From End of List

Medium. https://leetcode.com/problems/remove-nth-node-from-end-of-list/

You are given `head`, the first node of a singly linked list, and an integer n. Remove the nth node from the end of the list and return the first node of the result. When n is 1, the last node is removed. The value of n is always valid.

For example, the list 1 → 2 → 3 → 4 → 5 with n = 2 becomes 1 → 2 → 3 → 5.

Each node is a `ListNode` with the fields `val` and `next`, and `ListNode(val, next)` builds a new node. Start with `def removeNthFromEnd(head, n):`

Your Python solution

Class 8, Problem B, continued: #19 Remove Nth Node From End of List.

Time complexity:

Auxiliary space:

Estimate the memory for a list of L = 30 nodes, which is LeetCode's limit. LeetCode's values run from 0 to 100, so they are shared small integers.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 9, Problem A

Class 9: Recursion and divide and conquer. Problem A, the easier problem.

#509 Fibonacci Number

Easy. https://leetcode.com/problems/fibonacci-number/

The Fibonacci numbers start with F(0) = 0 and F(1) = 1, and every later number is the sum of the two before it: F(n) = F(n − 1) + F(n − 2). Given n, where 0 ≤ n ≤ 30, return F(n).

For example, n = 2 gives 1, n = 3 gives 2, and n = 4 gives 3.

Start with `def fib(n):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 30, which is LeetCode's limit.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 9, Problem B

Class 9: Recursion and divide and conquer. Problem B, the harder problem.

#50 Pow(x, n)

Medium. https://leetcode.com/problems/powx-n/

Compute x raised to the power n, where x is a float and n is an integer that can be negative, zero or positive. The exponent n lies between -2^{31} and $2^{31} - 1$, so a loop that multiplies x by itself n times is far too slow. LeetCode also guarantees that $-100 < x < 100$, that x is not zero whenever n is zero or negative, and that the answer lies between -10^4 and 10^4.

For example, x = 2.0 and n = 10 give 1024.0, and x = 2.0 and n = −2 give 0.25.

Start with `def myPow(x, n):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for $n = 2^{31}$, so that the recursion is about 32 calls deep.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 10, Problem A

Class 10: Sorting I: insertion sort and merge sort. Problem A, the easier problem.

#1331 Rank Transform of an Array

Easy. https://leetcode.com/problems/rank-transform-of-an-array/

You are given a list of integers, `arr`. Replace each element with its rank: the smallest value has rank 1, the next larger distinct value has rank 2, and so on, and equal values share the same rank. Return the list of ranks.

For example, `arr = [40, 10, 20, 30]` gives `[4, 1, 2, 3]`, and `arr = [37, 12, 28, 9, 100, 56, 80, 5, 12]` gives `[5, 3, 4, 2, 8, 6, 7, 1, 3]`.

Start with `def arrayRankTransform(arr):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 100,000 distinct integers, all above 256.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 10, Problem B

Class 10: Sorting I: insertion sort and merge sort. Problem B, the harder problem.

#912 Sort an Array

Medium. https://leetcode.com/problems/sort-an-array/

You are given a list of integers, `nums`. Sort it in ascending order and return it. You may not use any built-in sorting function. Your solution must run in O(n log n) time and use as little extra space as possible.

For example, `nums = [5, 2, 3, 1]` gives `[1, 2, 3, 5]`, and `nums = [5, 1, 1, 2, 0, 0]` gives `[0, 0, 1, 1, 2, 5]`.

Start with `def sortArray(nums):`

Your Python solution

Class 10, Problem B, continued: #912 Sort an Array.

Time complexity:

Auxiliary space:

Estimate the memory for n = 50,000 integers in random order, which is LeetCode's limit.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 11, Problem A

Class 11: Sorting II: quicksort and counting sort. Problem A, the easier problem.

#1122 Relative Sort Array

Easy. https://leetcode.com/problems/relative-sort-array/

You are given two lists of integers, `arr1` and `arr2`. The values in `arr2` are distinct, and every one of them also appears in `arr1`. Sort `arr1` so that its values appear in the same relative order as in `arr2`, and put the values that do not appear in `arr2` at the end, in ascending order. Every value lies between 0 and 1,000.

For example, `arr1 = [2, 3, 1, 3, 2, 4, 6, 7, 9, 2, 19]` and `arr2 = [2, 1, 4, 3, 9, 6]` give `[2, 2, 2, 1, 4, 3, 3, 9, 6, 7, 19]`.

Start with `def relativeSortArray(arr1, arr2):`

Your Python solution

Class 11, Problem A, continued: #1122 Relative Sort Array.

Time complexity:

Auxiliary space:

Estimate the memory for an `arr1` of 1,000 values and an `arr2` of 500 values, which are within LeetCode's limits, with values spread evenly from 0 to 1,000, so that about three quarters of them are above 256.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 11, Problem B

Class 11: Sorting II: quicksort and counting sort. Problem B, the harder problem.

#75 Sort Colors

Medium. https://leetcode.com/problems/sort-colors/

You are given a list, `nums`, of n objects colored red, white or blue, written as 0, 1 and 2. Sort the list in place so that objects of the same color are next to each other, with the colors in the order red, white, blue. You may not use a library sort. LeetCode asks for a single pass that uses only constant extra space.

For example, `nums = [2, 0, 2, 1, 1, 0]` becomes `[0, 0, 1, 1, 2, 2]`, and `nums = [2, 0, 1]` becomes `[0, 1, 2]`.

The function returns nothing; it changes `nums` itself. Start with `def sortColors(nums):`

Your Python solution

Class 11, Problem B, continued: #75 Sort Colors.

Time complexity:

Auxiliary space:

Estimate the memory for n = 300 values, which is LeetCode's limit.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 12, Problem A

Class 12: Binary search. Problem A, the easier problem.

#704 Binary Search

Easy. https://leetcode.com/problems/binary-search/

You are given a list of distinct integers, `nums`, sorted in ascending order, and an integer, `target`. Return the index of `target`, or −1 if it is not in the list. Your solution must run in O(log n) time.

For example, with `nums = [-1, 0, 3, 5, 9, 12]`, `target = 9` gives 4 and `target = 2` gives −1.

Start with `def search(nums, target):`

Your Python solution

Class 12, Problem A, continued: #704 Binary Search.

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 sorted distinct integers, which is LeetCode's limit, none of them between −5 and 256.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 12, Problem B

Class 12: Binary search. Problem B, the harder problem.

#34 Find First and Last Position of Element in Sorted Array

Medium. https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/

You are given a list of integers, `nums`, sorted in non-decreasing order, so it may contain repeated values, and an integer, `target`. Return `[first, last]`, the first and the last index at which `target` appears, or `[-1, -1]` if it does not appear. Your solution must run in O(log n) time.

For example, with `nums = [5, 7, 7, 8, 8, 10]`, `target = 8` gives `[3, 4]` and `target = 6` gives `[-1, -1]`.

Start with `def searchRange(nums, target):`

Your Python solution

Class 12, Problem B, continued: #34 Find First and Last Position of Element in Sorted Array.

Time complexity:

Auxiliary space:

Estimate the memory for n = 100,000 sorted integers above 256, which is LeetCode's limit, including a block of 1,000 copies of `target`.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 13, Problem A

Class 13: Trees and traversal. Problem A, the easier problem.

#104 Maximum Depth of Binary Tree

Easy. https://leetcode.com/problems/maximum-depth-of-binary-tree/

You are given `root`, the root node of a binary tree. Return the tree's maximum depth, which is the number of nodes along the longest path from the root down to a leaf.

For example, `root = [3, 9, 20, null, null, 15, 7]` gives 3, and `root = [1, null, 2]` gives 2. LeetCode writes a tree as a list of its values level by level, with `null` for a missing child.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def maxDepth(root):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 nodes in a balanced tree, which is 14 levels tall. LeetCode's values run from −100 to 100, so most of them are shared small integers.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 13, Problem B

Class 13: Trees and traversal. Problem B, the harder problem.

#102 Binary Tree Level Order Traversal

Medium. https://leetcode.com/problems/binary-tree-level-order-traversal/

You are given `root`, the root node of a binary tree. Return its level order traversal: a list of lists, in which list i holds the values of all the nodes at depth i, from left to right.

For example, `root = [3, 9, 20, null, null, 15, 7]` gives `[[3], [9, 20], [15, 7]]`.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def levelOrder(root):`

Your Python solution

Class 13, Problem B, continued: #102 Binary Tree Level Order Traversal.

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 nodes in a complete tree, which is 14 levels tall. That is five times LeetCode's limit of 2,000 nodes. LeetCode's values run from −1,000 to 1,000, so count each value as a new integer.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 14, Problem A

Class 14: Binary search trees. Problem A, the easier problem.

#700 Search in a Binary Search Tree

Easy. https://leetcode.com/problems/search-in-a-binary-search-tree/

You are given `root`, the root node of a binary search tree, and a value, `val`. Return the node whose value equals `val`, or `None` if there is no such node. The node you return stands for the whole subtree below it.

For example, with `root = [4, 2, 7, 1, 3]`, `val = 2` gives the subtree `[2, 1, 3]`, and `val = 5` gives `None`.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def searchBST(root, val):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 5,000 nodes, which is LeetCode's limit, in a balanced tree. LeetCode's values run from 1 to 10^7, so count each value as a new integer.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 14, Problem B

Class 14: Binary search trees. Problem B, the harder problem.

#98 Validate Binary Search Tree

Medium. https://leetcode.com/problems/validate-binary-search-tree/

You are given `root`, the root node of a binary tree. Return `True` if it is a valid binary search tree, and `False` otherwise. A tree is valid when, for every node, all the values in its left subtree are strictly smaller than the node's value and all the values in its right subtree are strictly larger.

For example, `root = [2, 1, 3]` gives `True`, and `root = [5, 1, 4, null, null, 3, 6]` gives `False`, because the root's right child, 4, is smaller than 5.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def isValidBST(root):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 nodes, which is LeetCode's limit, in a balanced valid tree, 14 levels tall. LeetCode's values can be as large as 2^{31} in size, so count each value as a new integer.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 15, Problem A

Class 15: Balanced search trees. Problem A, the easier problem.

#110 Balanced Binary Tree

Easy. https://leetcode.com/problems/balanced-binary-tree/

You are given `root`, the root node of a binary tree. Return `True` if the tree is height-balanced and `False` otherwise. A tree is height-balanced when, at every node, the heights of the left and right subtrees differ by at most one.

For example, `root = [3, 9, 20, null, null, 15, 7]` gives `True`, `root = [1, 2, 2, 3, 3, null, null, 4, 4]` gives `False`, and `root = []` gives `True`.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def isBalanced(root):`

Your Python solution

Class 15, Problem A, continued: #110 Balanced Binary Tree.

Time complexity:

Auxiliary space:

Estimate the memory for n = 5,000 nodes, which is LeetCode's limit, in a balanced tree, 13 levels tall. LeetCode's values run from −10,000 to 10,000, so count each value as a new integer.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 15, Problem B

Class 15: Balanced search trees. Problem B, the harder problem.

#1382 Balance a Binary Search Tree

Medium. https://leetcode.com/problems/balance-a-binary-search-tree/

You are given `root`, the root node of a binary search tree. Return a balanced binary search tree with the same values. A binary search tree is balanced when, at every node, the depths of the two subtrees differ by at most one. If more than one answer is possible, return any of them.

For example, `root = [1, null, 2, null, 3, null, 4]`, a chain, can give `[2, 1, 3, null, null, null, 4]`; `[3, 1, 4, null, 2]` is also correct. And `root = [2, 1, 3]` gives `[2, 1, 3]`.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def balanceBST(root):`

Your Python solution

Class 15, Problem B, continued: #1382 Balance a Binary Search Tree.

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 nodes, which is LeetCode's limit, arranged in a single chain to the right, as in the first example. LeetCode's values run from 1 to 100,000, so count each value as a new integer.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 16, Problem A

Class 16: Heaps, priority queues and heap sort. Problem A, the easier problem.

#703 Kth Largest Element in a Stream

Easy. https://leetcode.com/problems/kth-largest-element-in-a-stream/

Design a class `KthLargest`. It is created with an integer, `k`, and an initial list of integers, `nums`. Its method `add(val)` adds `val` to the stream of values and returns the kth largest value of everything seen so far, counting repeated values separately.

For example, after `KthLargest(3, [4, 5, 8, 2])`, the calls `add(3)`, `add(5)`, `add(10)`, `add(9)` and `add(4)` return 4, 5, 5, 8 and 8.

Start with `class KthLargest:` and write the methods `__init__(self, k, nums)` and `add(self, val)`.

Your Python solution

Class 16, Problem A, continued: #703 Kth Largest Element in a Stream.

Time complexity:

Auxiliary space:

Estimate the memory for k = 10,000, an initial list of 10,000 values and 10,000 calls to `add`, which are LeetCode's limits. The values lie between −10,000 and 10,000, and the caller creates each one.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 16, Problem B

Class 16: Heaps, priority queues and heap sort. Problem B, the harder problem.

#347 Top K Frequent Elements

Medium. https://leetcode.com/problems/top-k-frequent-elements/

You are given a list of integers, `nums`, and an integer, `k`. Return the k values that appear most often, in any order. The answer is guaranteed to be unique, and your solution should be faster than O(n log n).

For example, `nums = [1, 1, 1, 2, 2, 3]` and `k = 2` give `[1, 2]`.

Start with `def topKFrequent(nums, k):`

Your Python solution

Class 16, Problem B, continued: #347 Top K Frequent Elements.

Time complexity:

Auxiliary space:

Estimate the memory for n = 100,000 values, which is LeetCode's limit, made of 20,000 distinct values between −10,000 and 10,000 that each appear five times, and k = 100. Assume that the copies of a value are the same integer object.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 17, Problem A

Class 17: Graphs and depth-first search. Problem A, the easier problem.

#733 Flood Fill

Easy. https://leetcode.com/problems/flood-fill/

`image` is a grid of integers, a picture of pixel colors. Starting from the pixel at row `sr` and column `sc`, change to `color` that pixel and every pixel connected to it by moves up, down, left or right through pixels of the same starting color. Return the changed image.

For example, `image = [[1, 1, 1], [1, 1, 0], [1, 0, 1]]`, `sr = 1`, `sc = 1` and `color = 2` give `[[2, 2, 2], [2, 2, 0], [2, 0, 1]]`.

Start with `def floodFill(image, sr, sc, color):`

Your Python solution

Class 17, Problem A, continued: #733 Flood Fill.

Time complexity:

Auxiliary space:

Estimate the memory for a 50 × 50 grid, which is LeetCode's limit, so n = 2,500 cells, all of one color. This is the worst case.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 17, Problem B

Class 17: Graphs and depth-first search. Problem B, the harder problem.

#200 Number of Islands

Medium. https://leetcode.com/problems/number-of-islands/

You are given a grid of the characters `"1"`, for land, and `"0"`, for water. Count the islands. An island is a group of land cells connected horizontally or vertically, and the grid is surrounded by water on every side.

For example, `grid = [["1","1","0","0","0"], ["1","1","0","0","0"], ["0","0","1","0","0"], ["0","0","0","1","1"]]` gives 3.

Start with `def numIslands(grid):`

Your Python solution

Class 17, Problem B, continued: #200 Number of Islands.

Time complexity:

Auxiliary space:

Estimate the memory for a 300 × 300 grid, which is LeetCode's limit, so n = 90,000 cells, all land, which makes one island.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 18, Problem A

Class 18: Breadth-first search. Problem A, the easier problem.

#1971 Find if Path Exists in Graph

Easy. https://leetcode.com/problems/find-if-path-exists-in-graph/

A graph has n vertices, numbered 0 to n − 1, and a list of undirected edges. Return `True` if there is a path from `source` to `destination`, and `False` otherwise.

For example, `n = 3`, `edges = [[0, 1], [1, 2], [2, 0]]`, `source = 0` and `destination = 2` give `True`, while `n = 6`, `edges = [[0, 1], [0, 2], [3, 5], [5, 4], [4, 3]]`, `source = 0` and `destination = 5` give `False`.

Start with `def validPath(n, edges, source, destination):`

Your Python solution

Class 18, Problem A, continued: #1971 Find if Path Exists in Graph.

Time complexity:

Auxiliary space:

Estimate the memory for n = 100,000 vertices joined in a single long path, with edges [i, i + 1]. LeetCode's limit is 200,000 vertices.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 18, Problem B

Class 18: Breadth-first search. Problem B, the harder problem.

#994 Rotting Oranges

Medium. https://leetcode.com/problems/rotting-oranges/

A grid holds 0 for an empty cell, 1 for a fresh orange and 2 for a rotten orange. Every minute, each fresh orange next to a rotten orange, up, down, left or right, becomes rotten. Return the smallest number of minutes after which no fresh orange remains, or −1 if that can never happen.

For example, `grid = [[2, 1, 1], [1, 1, 0], [0, 1, 1]]` gives 4, `grid = [[2, 1, 1], [0, 1, 1], [1, 0, 1]]` gives −1, and `grid = [[0, 2]]` gives 0.

Start with `def orangesRotting(grid):`

Your Python solution

Class 18, Problem B, continued: #994 Rotting Oranges.

Time complexity:

Auxiliary space:

Estimate the memory for a 10 × 10 grid, which is LeetCode's limit, so n = 100 cells, with a single rotten orange in a corner and every other cell fresh.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 19, Problem A

Class 19: Topological sort. Problem A, the easier problem.

#207 Course Schedule

Medium. https://leetcode.com/problems/course-schedule/

There are `numCourses` courses, numbered 0 to `numCourses - 1`. Each entry `prerequisites[i] = [a, b]` means that you must take course b before course a. Return `True` if you can finish every course, which is possible exactly when the prerequisites contain no cycle, and `False` otherwise.

For example, `numCourses = 2` and `prerequisites = [[1, 0]]` give `True`, while `numCourses = 2` and `prerequisites = [[1, 0], [0, 1]]` give `False`.

Start with `def canFinish(numCourses, prerequisites):`

Your Python solution

Class 19, Problem A, continued: #207 Course Schedule.

Time complexity:

Auxiliary space:

Estimate the memory for V = 2,000 courses, which is LeetCode's limit, in one long chain of prerequisites, so E = 1,999.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 19, Problem B

Class 19: Topological sort. Problem B, the harder problem.

#210 Course Schedule II

Medium. https://leetcode.com/problems/course-schedule-ii/

The setup is the same as in Course Schedule: `prerequisites[i] = [a, b]` means that you must take course b before course a. Return one order in which you can take all the courses. If no such order exists, because the prerequisites contain a cycle, return an empty list.

For example, `numCourses = 4` and `prerequisites = [[1, 0], [2, 0], [3, 1], [3, 2]]` give `[0, 1, 2, 3]`; `[0, 2, 1, 3]` is also correct.

Start with `def findOrder(numCourses, prerequisites):`

Your Python solution

Class 19, Problem B, continued: #210 Course Schedule II.

Time complexity:

Auxiliary space:

Estimate the memory for V = 2,000 courses in one long chain of prerequisites, so E = 1,999.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 20, Problem A

Class 20: Shortest paths I: Dijkstra's algorithm. Problem A, the easier problem.

#743 Network Delay Time

Medium. https://leetcode.com/problems/network-delay-time/

A network has n nodes, labeled 1 to n. Each entry `times[i] = [u, v, w]` is a directed edge: a signal sent from u reaches v after w units of time. A signal is sent from node k. Return the time it takes for all n nodes to receive the signal, or −1 if some node can never receive it.

For example, `times = [[2, 1, 1], [2, 3, 1], [3, 4, 1]]`, `n = 4` and `k = 2` give 2.

Start with `def networkDelayTime(times, n, k):`

Your Python solution

Class 20, Problem A, continued: #743 Network Delay Time.

Time complexity:

Auxiliary space:

Estimate the memory for V = 100 nodes and E = 6,000 edges, which are LeetCode's limits, with random edges whose weights run from 1 to 100.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 20, Problem B

Class 20: Shortest paths I: Dijkstra's algorithm. Problem B, the harder problem.

#1631 Path With Minimum Effort

Medium. https://leetcode.com/problems/path-with-minimum-effort/

`heights` is a grid of integers. You start at the top-left cell and want to reach the bottom-right cell, moving up, down, left or right. The effort of a route is the largest absolute difference in height between two consecutive cells on the route. Return the smallest effort of any route.

For example, `heights = [[1, 2, 2], [3, 8, 2], [5, 3, 5]]` gives 2.

Start with `def minimumEffortPath(heights):`

Your Python solution

Class 20, Problem B, continued: #1631 Path With Minimum Effort.

Time complexity:

Auxiliary space:

Estimate the memory for a 100 × 100 grid, which is LeetCode's limit, so n = 10,000 cells, that is flat, with every height equal to 1 except at the goal, so that the search explores every cell at effort 0 before it reaches the goal.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 21, Problem A

Class 21: Shortest paths II: Bellman-Ford and Floyd-Warshall. Problem A, the easier problem.

#1334 Find the City With the Smallest Number of Neighbors at a Threshold Distance

Medium. https://leetcode.com/problems/find-the-city-with-the-smallest-number-of-neighbors-at-a-threshold-distance/

There are n cities, numbered 0 to n − 1. Each entry `edges[i] = [from, to, weight]` is a road in both directions between two cities, with the given length. The distance between two cities is the length of the shortest route between them. Return the city from which the fewest other cities lie within `distanceThreshold`. If several cities tie, return the one with the largest number.

For example, `n = 4`, `edges = [[0, 1, 3], [1, 2, 1], [1, 3, 4], [2, 3, 1]]` and `distanceThreshold = 4` give 3: cities 0 and 3 each reach two others, and 3 is the larger number.

Start with `def findTheCity(n, edges, distanceThreshold):`

Your Python solution

Class 21, Problem A, continued: #1334 Find the City With the Smallest Number of Neighbors at a Threshold Distance.

Time complexity:

Auxiliary space:

Estimate the memory for n = 100 cities, which is LeetCode's limit, with a road between every pair, 4,950 roads, whose lengths run from 300 to 10,000.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 21, Problem B

Class 21: Shortest paths II: Bellman-Ford and Floyd-Warshall. Problem B, the harder problem.

#787 Cheapest Flights Within K Stops

Medium. https://leetcode.com/problems/cheapest-flights-within-k-stops/

There are n cities. Each entry `flights[i] = [from, to, price]` is a one-way flight with the given price. Return the cheapest price from city `src` to city `dst` using at most k stops in between, or −1 if no such route exists.

For example, with `n = 4` and `flights = [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]]`, `src = 0`, `dst = 3` and `k = 1` give 700, by flying 0 to 1 to 3. The route 0 to 1 to 2 to 3 costs only 400, but it makes two stops.

Start with `def findCheapestPrice(n, flights, src, dst, k):`

Your Python solution

Class 21, Problem B, continued: #787 Cheapest Flights Within K Stops.

Time complexity:

Auxiliary space:

Estimate the memory for n = 100 cities, which is LeetCode's limit, with 4,950 flights whose prices run from 300 to 10,000, and k = 99.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 22, Problem A

Class 22: Union-find. Problem A, the easier problem.

#547 Number of Provinces

Medium. https://leetcode.com/problems/number-of-provinces/

There are n cities. `isConnected` is an n × n grid in which `isConnected[i][j]` is 1 if cities i and j are directly connected and 0 otherwise. A province is a group of cities connected to each other directly or through other cities. Return the number of provinces.

For example, `isConnected = [[1, 1, 0], [1, 1, 0], [0, 0, 1]]` gives 2, and `isConnected = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]` gives 3.

Start with `def findCircleNum(isConnected):`

Your Python solution

Class 22, Problem A, continued: #547 Number of Provinces.

Time complexity:

Auxiliary space:

Estimate the memory for n = 200 cities, which is LeetCode's limit, with no connections except each city to itself. This is the worst case for the scan.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 22, Problem B

Class 22: Union-find. Problem B, the harder problem.

#684 Redundant Connection

Medium. https://leetcode.com/problems/redundant-connection/

A tree with n nodes, labeled 1 to n, had one extra edge added to it. The input is the resulting list of n edges, which contains exactly one cycle. Return an edge whose removal leaves a tree. If more than one edge works, return the one that appears last in the input.

For example, `edges = [[1, 2], [1, 3], [2, 3]]` gives `[2, 3]`, and `edges = [[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]` gives `[1, 4]`.

Start with `def findRedundantConnection(edges):`

Your Python solution

Class 22, Problem B, continued: #684 Redundant Connection.

Time complexity:

Auxiliary space:

Estimate the memory for n = 1,000 edges, which is LeetCode's limit, forming a path 1–2–…–1,000, with the edge [1, 1,000] that closes the cycle last.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 23, Problem A

Class 23: Minimum spanning trees. Problem A, the easier problem.

#1584 Min Cost to Connect All Points

Medium. https://leetcode.com/problems/min-cost-to-connect-all-points/

You are given a list of points on a plane, `points[i] = [x, y]`. Connecting two points costs their Manhattan distance, $|x_1 - x_2| + |y_1 - y_2|$. Return the smallest total cost that connects all the points, so that there is exactly one path between any two of them.

For example, `points = [[0, 0], [2, 2], [3, 10], [5, 2], [7, 0]]` gives 20, and `points = [[3, 12], [-2, 5], [-4, 1]]` gives 18.

Start with `def minCostConnectPoints(points):`

Your Python solution

Class 23, Problem A, continued: #1584 Min Cost to Connect All Points.

Time complexity:

Auxiliary space:

Estimate the memory for n = 1,000 points, which is LeetCode's limit, with coordinates as large as 10^6, so count each coordinate as a new integer.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 23, Problem B

Class 23: Minimum spanning trees. Problem B, the harder problem.

#1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

Hard. https://leetcode.com/problems/find-critical-and-pseudo-critical-edges-in-minimum-spanning-tree/

You are given a connected, undirected graph with n vertices and a list of weighted edges, `edges[i] = [a, b, weight]`. A minimum spanning tree connects every vertex, without cycles, at the smallest total weight. An edge is critical if deleting it from the graph would raise that smallest weight. An edge is pseudo-critical if it appears in some minimum spanning trees but not in all of them. Return `[critical, pseudo]`, the indexes of the critical edges and of the pseudo-critical edges, each in any order.

For example, `n = 4` and `edges = [[0, 1, 1], [1, 2, 1], [2, 3, 1], [0, 3, 1]]` give `[[], [0, 1, 2, 3]]`: all four edges weigh the same, so any three of them form a minimum spanning tree, and no edge is in every one.

Start with `def findCriticalAndPseudoCriticalEdges(n, edges):`

Your Python solution

Class 23, Problem B, continued: #1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree.

Time complexity:

Auxiliary space:

Estimate the memory for n = 100 vertices and 200 edges, which are LeetCode's limits, with weights spread from 1 to 1,000.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 24, Problem A

Class 24: Backtracking, P and NP. Problem A, the easier problem.

#78 Subsets

Medium. https://leetcode.com/problems/subsets/

You are given a list of distinct integers, `nums`. Return every possible subset of the list, in any order, without repeating a subset.

For example, `nums = [1, 2, 3]` gives `[[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]`.

Start with `def subsets(nums):`

Your Python solution

Class 24, Problem A, continued: #78 Subsets.

Time complexity:

Auxiliary space:

Estimate the memory for n = 10, which is LeetCode's limit, so there are 1,024 subsets with 5,120 entries in all. LeetCode's values run from −10 to 10.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 24, Problem B

Class 24: Backtracking, P and NP. Problem B, the harder problem.

#51 N-Queens

Hard. https://leetcode.com/problems/n-queens/

Place n queens on an n × n chessboard so that no two queens attack each other, which means that no two share a row, a column or a diagonal. Return every distinct solution, each written as a list of n strings with `"Q"` for a queen and `"."` for an empty square.

For example, n = 4 gives `[[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]`.

Start with `def solveNQueens(n):`

Your Python solution

Class 24, Problem B, continued: #51 N-Queens.

Time complexity:

Auxiliary space:

Estimate the memory for n = 8, which has 92 solutions, each a list of 8 strings of 8 characters.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Class 25: Tries. Problem A, the easier problem.

#208 Implement Trie (Prefix Tree)

Medium. https://leetcode.com/problems/implement-trie-prefix-tree/

Implement a trie. `insert(word)` stores a word; `search(word)` returns `True` if that exact word was inserted; and `startsWith(prefix)` returns `True` if any inserted word begins with `prefix`. Every string uses lowercase letters only.

For example, after `insert("apple")`, the call `search("apple")` returns `True`, `search("app")` returns `False` and `startsWith("app")` returns `True`; after `insert("app")`, `search("app")` returns `True`.

Start with `class Trie:` and write the methods `__init__(self)`, `insert(self, word)`, `search(self, word)` and `startsWith(self, prefix)`.

Your Python solution

Class 25, Problem A, continued: #208 Implement Trie (Prefix Tree).

Time complexity:

Auxiliary space:

Estimate the memory for 10,000 random words of 8 lowercase letters each, 80,000 letters in all, created by the caller.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 25, Problem B

Class 25: Tries. Problem B, the harder problem.

#212 Word Search II

Hard. https://leetcode.com/problems/word-search-ii/

You are given an m × n board of letters and a list of distinct words. Return every word from the list that can be spelled by a path of neighboring cells, moving horizontally or vertically. A path may use each cell at most once within the same word.

For example, `board = [["o","a","a","n"], ["e","t","a","e"], ["i","h","k","r"], ["i","f","l","v"]]` and `words = ["oath","pea","eat","rain"]` give `["eat", "oath"]`.

Start with `def findWords(board, words):`

Your Python solution

Class 25, Problem B, continued: #212 Word Search II.

Time complexity:

Auxiliary space:

Estimate the memory for 10,000 random words of 8 letters each, stored in a trie before the search starts, on a 12 × 12 board of random letters.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 26, Problem A

Class 26: Greedy algorithms and intervals. Problem A, the easier problem.

#55 Jump Game

Medium. https://leetcode.com/problems/jump-game/

`nums[i]` is the longest jump you can make from position i. Starting at index 0, return `True` if you can reach the last index, and `False` otherwise.

For example, `nums = [2, 3, 1, 1, 4]` gives `True`, and `nums = [3, 2, 1, 0, 4]` gives `False`.

Start with `def canJump(nums):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 10,000 values, which is LeetCode's limit, all equal to 1, so that the reach grows one step at a time.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 26, Problem B

Class 26: Greedy algorithms and intervals. Problem B, the harder problem.

#135 Candy

Hard. https://leetcode.com/problems/candy/

n children stand in a line, and `ratings[i]` is the rating of child i. Give each child at least one candy, and give a child with a higher rating than a neighbor more candies than that neighbor. Return the smallest total number of candies.

For example, `ratings = [1, 0, 2]` gives 5, with candies 2, 1 and 2, and `ratings = [1, 2, 2]` gives 4, with candies 1, 2 and 1.

Start with `def candy(ratings):`

Your Python solution

Class 26, Problem B, continued: #135 Candy.

Time complexity:

Auxiliary space:

Estimate the memory for n = 50,000 children, which is LeetCode's limit, with ratings that strictly increase. This is the worst case for the size of the candy counts, which then run 1, 2, …, n.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 27, Problem A

Class 27: Dynamic programming: one dimension. Problem A, the easier problem.

#198 House Robber

Medium. https://leetcode.com/problems/house-robber/

`nums[i]` is the amount of money in house i along a street. You may not rob two neighboring houses. Return the largest amount you can rob.

For example, `nums = [1, 2, 3, 1]` gives 4, by robbing houses 0 and 2, and `nums = [2, 7, 9, 3, 1]` gives 12, by robbing houses 0, 2 and 4.

Start with `def rob(nums):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 100 houses, which is LeetCode's limit, with values up to 400.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 27, Problem B

Class 27: Dynamic programming: one dimension. Problem B, the harder problem.

#322 Coin Change

Medium. https://leetcode.com/problems/coin-change/

`coins` is a list of coin values, with an unlimited supply of each, and `amount` is a target. Return the smallest number of coins that add up to exactly `amount`, or −1 if no combination does.

For example, `coins = [1, 2, 5]` and `amount = 11` give 3, from 5 + 5 + 1, and `coins = [2]` and `amount = 3` give −1.

Start with `def coinChange(coins, amount):`

Your Python solution

Class 27, Problem B, continued: #322 Coin Change.

Time complexity:

Auxiliary space:

Estimate the memory for `amount = 10,000`, which is LeetCode's limit, with `coins = [1, 2, 5]`.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 28, Problem A

Class 28: Dynamic programming: sequences. Problem A, the easier problem.

#300 Longest Increasing Subsequence

Medium. https://leetcode.com/problems/longest-increasing-subsequence/

You are given a list of integers, `nums`. Return the length of its longest strictly increasing subsequence. A subsequence keeps the order of the numbers, but its numbers need not be next to each other.

For example, `nums = [10, 9, 2, 5, 3, 7, 101, 18]` gives 4, for example from 2, 3, 7, 101.

Start with `def lengthOfLIS(nums):`

Your Python solution

Time complexity:

Auxiliary space:

Estimate the memory for n = 2,500 values, which is LeetCode's limit, strictly increasing. This is the worst case for the values in `dp`, which then run 1, 2, …, n.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 28, Problem B

Class 28: Dynamic programming: sequences. Problem B, the harder problem.

#1143 Longest Common Subsequence

Medium. https://leetcode.com/problems/longest-common-subsequence/

You are given two strings, `text1` and `text2`. Return the length of their longest common subsequence, the longest sequence of characters that appears in both strings in the same order, though not necessarily next to each other. Return 0 if they have none.

For example, `text1 = "abcde"` and `text2 = "ace"` give 3, from `"ace"`, and `text1 = "abc"` and `text2 = "def"` give 0.

Start with `def longestCommonSubsequence(text1, text2):`

Your Python solution

Class 28, Problem B, continued: #1143 Longest Common Subsequence.

Time complexity:

Auxiliary space:

Estimate the memory for two random strings of 1,000 lowercase letters each, which is LeetCode's limit.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 29, Problem A

Class 29: Dynamic programming: two-dimensional tables. Problem A, the easier problem.

#72 Edit Distance

Medium. https://leetcode.com/problems/edit-distance/

You are given two words, `word1` and `word2`. Return the smallest number of single-character operations, each an insertion, a deletion or a replacement, that turn `word1` into `word2`.

For example, `word1 = "horse"` and `word2 = "ros"` give 3: horse becomes rorse, then rose, then ros.

Start with `def minDistance(word1, word2):`

Your Python solution

Class 29, Problem A, continued: #72 Edit Distance.

Time complexity:

Auxiliary space:

Estimate the memory for two random words of 500 lowercase letters each, which is LeetCode's limit.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 29, Problem B

Class 29: Dynamic programming: two-dimensional tables. Problem B, the harder problem.

#10 Regular Expression Matching

Hard. https://leetcode.com/problems/regular-expression-matching/

Implement pattern matching in which `"."` matches any single character and `"*"` matches zero or more copies of the character just before it. Return `True` if the pattern `p` matches the whole string `s`, and `False` otherwise.

For example, `s = "aa"` and `p = "a"` give `False`, `s = "aa"` and `p = "a*"` give `True`, and `s = "ab"` and `p = ".*"` give `True`.

Start with `def isMatch(s, p):`

Your Python solution

Class 29, Problem B, continued: #10 Regular Expression Matching.

Time complexity:

Auxiliary space:

Estimate the memory for a string and a pattern of 20 characters each, which is LeetCode's limit.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 30, Problem A

Class 30: Choosing the pattern. Problem A, the easier problem.

#127 Word Ladder

Hard. https://leetcode.com/problems/word-ladder/

`beginWord` and `endWord` are words of the same length, and `wordList` is a list of allowed words. A move changes exactly one letter, and every word you reach must be in `wordList`. Return the number of words in the shortest sequence of moves from `beginWord` to `endWord`, counting both ends, or 0 if no such sequence exists.

For example, `beginWord = "hit"`, `endWord = "cog"` and `wordList = ["hot","dot","dog","lot","log","cog"]` give 5: hit, hot, dot, dog, cog.

Start with `def ladderLength(beginWord, endWord, wordList):`

Your Python solution

Class 30, Problem A, continued: #127 Word Ladder.

Time complexity:

Auxiliary space:

Estimate the memory for N = 3,125 words, every 5-letter word made from the letters a to e, with L = 5, `beginWord = "aaaaa"` and `endWord = "eeeee"`.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.

Name _____ Date _____ Class 30, Problem B

Class 30: Choosing the pattern. Problem B, the harder problem.

#295 Find Median from Data Stream

Hard. https://leetcode.com/problems/find-median-from-data-stream/

Design a class `MedianFinder`. Its method `addNum(num)` adds an integer from a stream of data, and `findMedian()` returns the median of all the numbers so far: the middle value if the count is odd, or the mean of the two middle values if the count is even.

For example, after `addNum(1)` and `addNum(2)`, `findMedian()` returns 1.5; after `addNum(3)`, it returns 2.0.

Start with `class MedianFinder:` and write the methods `__init__(self)`, `addNum(self, num)` and `findMedian(self)`.

Your Python solution

Class 30, Problem B, continued: #295 Find Median from Data Stream.

Time complexity:

Auxiliary space:

Estimate the memory for n = 50,000 numbers added, which is LeetCode's limit.

Memory used by the input, in bytes:

Memory used by your auxiliary data, in bytes:

Total memory, in bytes:

If you asked for help, write your two failed attempts here first.

1.

2.